

PRAKTIKUM SISTEM OPERASI

Modul 5: Eksplorasi Proses Xinu



Oleh:

Reza Afiansyah Wibowo

2311104062

PROGRAM STUDI S1 REKAYASA PERANGKAT LUNAK

DIREKTORAT KAMPUS PURWOKERTO

UNIVERSITAS TELKOM

2026

JURNAL

Soal 1 [10 Poin] — Pertanyaan Konsep PCB Xinu

a. Jumlah maksimum proses pada Xinu

Jumlah maksimum proses ditentukan oleh konstanta NPROC yang didefinisikan pada file konfigurasi kernel (conf.h / kernel.h). Pada konfigurasi standar Xinu yang umum dipakai di praktikum, nilai NPROC adalah 100 proses. Nilai ini dapat diverifikasi dengan menjalankan perintah berikut pada source code:
`grep -rn "NPROC" /xinu/include/`

b. Panjang maksimal nama proses

Panjang maksimal nama proses ditentukan oleh konstanta PNMLEN, dengan nilai standar 16 karakter (termasuk karakter null terminator).

c. Nilai prioritas awal saat proses dibuat

Pada Xinu, prioritas tidak memiliki satu nilai default yang tetap untuk semua proses — nilai prioritas diberikan sebagai parameter (prio) saat pemanggilan fungsi create(). Namun, proses-proses bawaan sistem seperti Main process dan ps yang dibuat melalui shell umumnya diberi prioritas 20, sehingga nilai 20 sering disebut sebagai prioritas standar pada praktikum ini.

d. Total state pada Xinu

Xinu memiliki 8 (delapan) state proses, yaitu:

1. PR_FREE - slot proses kosong / belum digunakan
2. PR_CURR - proses sedang dieksekusi oleh CPU
3. PR_READY - proses siap dijalankan, menunggu di ready list
4. PR_RECV - proses menunggu menerima pesan (message)
5. PR_SLEEP - proses tertidur untuk jangka waktu tertentu
6. PR_SUSP - proses dalam keadaan suspend
7. PR_WAIT - proses menunggu semaphore
8. PR_RECTIM - proses menunggu pesan dengan batas waktu (timeout)

Soal 2 [20 Poin] — Komentar pada xsh_ps.c

File xsh_ps.c berisi source code perintah ps, biasanya terletak pada direktori shell/ di source tree Xinu. File ini dapat dicari dengan perintah:
`find / -name "xsh_ps.c" 2>/dev/null`

Bagian akhir file ini umumnya berupa loop yang membaca tabel proses (proctab) dan mencetak setiap kolom informasinya. Berikut contoh struktur 20 baris terakhir yang sudah diberi komentar

```
/* Nama - NIM */
```

```
/* Cetak header tabel: nama kolom statistik proses */
```

```

printf("Pid Name      State   Prio Ppid");
printf(" Stack Base Stack Ptr  Stack Size\n");
printf("---- -");
printf("-----\n");

/* Iterasi seluruh slot pada tabel proses (proctab) */
for (i = 0; i < NPROC; i++) {
    prptr = &proctab[i];

    /* Lewati slot yang sedang kosong (tidak terpakai) */
    if (prptr->prstate == PR_FREE) {
        continue;
    }

    /* Cetak satu baris informasi proses: pid, nama,
     * state (dikonversi ke string), prioritas, ppid,
     * alamat dasar stack, pointer stack, ukuran stack */
    printf("%3d %-16s %-9s %4d %4d 0x%08X 0x%08X %10d\n",
        i, prptr->prname, stateStr[prptr->prstate],
        prptr->prprio, prptr->prparent,
        (uint32)prptr->prstkbase,
        (uint32)prptr->prstkptr, prptr->prstklen);
}
return OK;
}

```

Soal 3 [35 Poin] — Modifikasi ps: Kolom Msg dan Content

Modifikasi menambahkan dua kolom baru pada output ps: **Msg** (jumlah pesan yang ada pada proses) dan **Content** (isi pesan tersebut). Pada Xinu standar, setiap proses hanya menyimpan satu pesan tertunda melalui field **prhasmsg** (flag ada/tidaknya pesan) dan **prmsg** (isi pesan). Sesuaikan nama field berikut dengan struct **procent** pada **proc.h** milikmu jika berbeda.

Modifikasi pada xsh_ps.c

```
/* Nama - NIM */
```

```

/* Tambahkan header kolom Msg dan Content */
printf("Pid Name      State   Prio Ppid");

```

```

printf(" Stack Base Stack Ptr  Stack Size Msg Content\n");
printf("---- -");
printf("----- -\n");

for (i = 0; i < NPROC; i++) {
    prptr = &proctab[i];
    if (prptr->prstate == PR_FREE) {
        continue;
    }

    /* prhasmsg = 1 jika ada pesan tertunda, 0 jika tidak */
    /* prmsg = isi pesan (disesuaikan tipe datanya) */
    printf("%3d %-16s %-9s %4d %4d 0x%08X 0x%08X %10d %3d %7d\n",
        i, prptr->prname, stateStr[prptr->prstate],
        prptr->prprio, prptr->prparent,
        (uint32)prptr->prstkbase,
        (uint32)prptr->prstkptr, prptr->prstklen,
        prptr->prhasmsg,
        prptr->prhasmsg ? prptr->prmsg : 0);
}
return OK;
}

```

Langkah pengujian: simpan perubahan, lalu kompilasi ulang Xinu, jalankan Backend VM, dan jalankan perintah \$ps untuk memastikan kolom Msg dan Content tampil sesuai contoh pada modul.

```

$ make clean
$ make
(jalankan Backend VM)
xsh $ ps

```

Soal 4 [35 Poin] — Modifikasi uptime dalam Satuan Menit

Perintah uptime pada Xinu menghitung lama waktu sejak sistem boot, biasanya berdasarkan variabel penghitung waktu global (misalnya clktime dalam satuan detik). Modifikasi berikut mengonversi nilai tersebut menjadi satuan menit. Sesuaikan nama variabel waktu dengan yang digunakan pada xsh_uptime.c aslimu.

Modifikasi pada xsh_uptime.c

/* Nama - NIM */

shellcmd xsh_uptime(int nargs, char *args[])

{

uint32 seconds;

uint32 minutes;

/* clktime menyimpan jumlah detik sejak Xinu boot */

seconds = clktime;

/* Konversi dari detik ke menit (pembagian bulat ke bawah) */

minutes = seconds / 60;

/* Tampilkan hasil uptime dalam satuan menit */

printf("Xinu has been up %d minute(s)\n", minutes);

return OK;

}

Langkah pengujian:

\$ make clean

\$ make

(jalankan Backend VM)

xsh \$ uptime